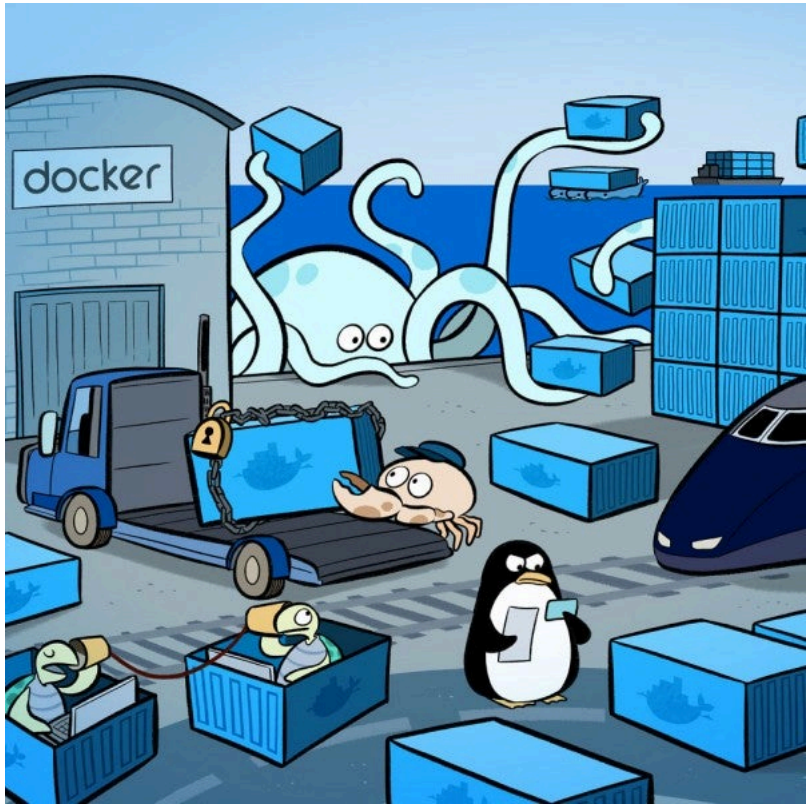
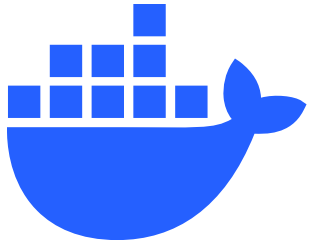
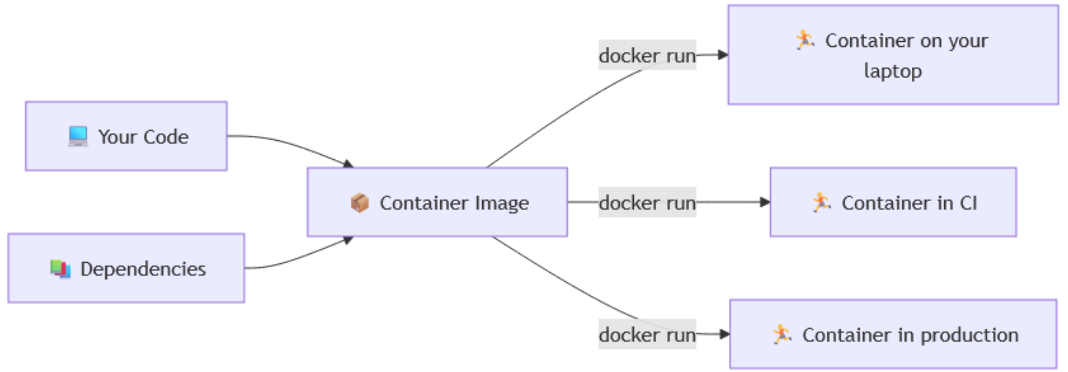
Docker Lab





> 📖 *Docker is an open-source platform for automating application deployment within containers.*

- Lightweight, portable execution environments
- Package applications with all dependencies
- Run consistently across different environments
- Share host OS kernel (no full OS needed)



Install & Verify

```
# Check version
docker --version

# Verify installation works
docker run hello-world

# Full system info
docker info
```

Platform Notes

Windows: WSL2 backend + Virtualization enabled in BIOS

macOS: Docker Desktop (uses a lightweight Linux VM internally)

Linux: Install Docker Engine directly — no VM needed, native performance

Client-Server Model

- **Docker Client** (`docker` CLI): translates commands into REST API calls sent to the Daemon
- **Docker Daemon** (`dockerd`): background process that builds, runs, and manages containers
- **Docker Host**: machine running the Daemon
  - holds images, containers, networks, volumes
- **Docker Registry**: remote store for images
  - **Docker Hub**: default public registry

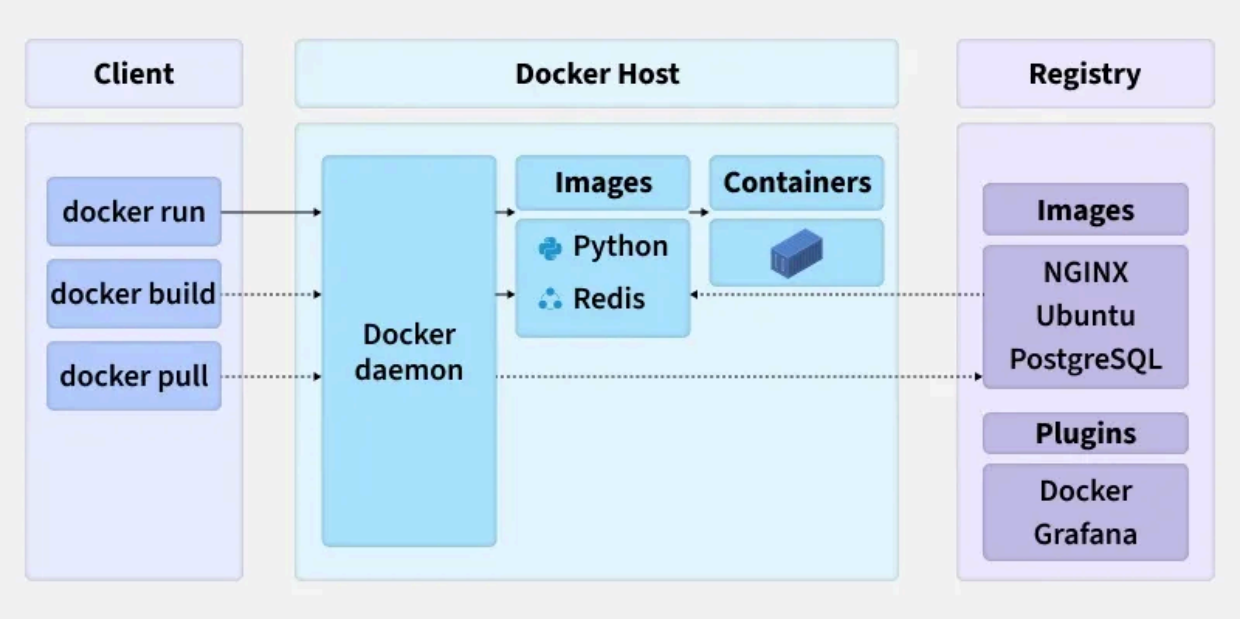
Core Objects

- **Dockerfile** → Recipe/instructions
- **Image** → Read-only layered template
- **Container** → Running instance of an image
- **Volume** → Persistent data storage
- **Network** → Communication between containers

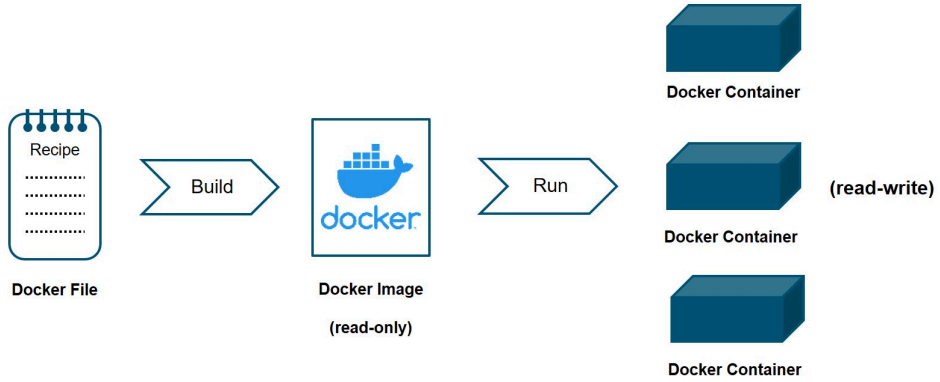
docker → Docker Run Example

`docker run nginx`

- *Client → Daemon checks local cache → pulls from Registry if missing → containerd + runc create isolated process with namespaces/cgroups*



docker → Dockerfile



> A **recipe** for building Docker images

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Key Instructions

Instruction	Purpose
`FROM`	Base image
`WORKDIR`	Set working directory
`COPY` / `ADD`	Copy files into image
`RUN`	Execute command during build
`ENV`	Set environment variable
`EXPOSE`	Document port (informational)
`CMD`	Default command to run
`ENTRYPOINT`	Fixed executable
`USER`	Run as specific user

```
docker build . -t myapp:1.0 # Build an image from Dockerfile

docker build -f Dockerfile.prod -t myapp:prod . # Build with specific Dockerfile

docker tag myapp:1.0 username/myapp:1.0 # Tag for registry

docker push username/myapp:1.0 # Push to registry

docker images # List local images
```

Naming Convention

- `image_name:tag`
- `username/repo:version`
- `registry.example.com/team/app:v2.1`

docker → Layer Caching

📖 *Each Dockerfile instruction creates a layer. Docker caches layers*

- *if nothing changed, it reuses the cached layer and skips re-running that step.*
- *Order instructions from least-changed to most-changed.*

No Cache Benefit

```
# ❌ Code copied first — cache breaks on every change
FROM python:3.11-slim
WORKDIR /app
COPY . .                                # changes every time
RUN pip install -r requirements.txt     # re-runs every build
```

Cache-Optimized

```
# ✅ Dependencies first — reinstall only when requirements.txt changes
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .                 # rarely changes → cached
RUN pip install -r requirements.txt     # skipped if unchanged
COPY . .                                # code changes here, not above
```

Rule: `COPY requirements.txt` (or `package*.json`) **before** `COPY . .`

- a code-only change won't invalidate the dependency install layer.

docker → Multi-Stage Builds

📖 *Separate build environment from runtime*

- *only the final artifact is copied into the production image*
- *Typical reduction: 1 GB → 150 MB.*

```
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install && npm run build

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Final image contains only runtime files

- *no dev tools, no source code, no build cache.*

Docker's Official Public Registry

Pull official images:

```
# Pull latest
docker pull nginx

# Pull specific version
docker pull python:3.11-slim
docker pull postgres:15-alpine
```

- databases (MySQL, MongoDB, PostgreSQL)
- runtimes (Python, Node.js, Java),
- servers (nginx, Apache)
- tools (Redis, RabbitMQ)

Publish your own image:

```
docker login
docker push username/myapp:1.0   # after tagging (see Building Images)
```

- **Public repos**: free, visible to everyone
- **Private repos**: access controlled (paid)
- **Official images**: verified and maintained by Docker
- **Verified publishers**: trusted third-party images

```
docker run -it ubuntu:latest bash # Run interactively

docker run -d -p 5000:5000 --name myapp myapp:1.0 # Run detached, port mapping
# Run with environment variables
docker run -e DATABASE_URL=postgresql://user:pass@db/app -p 3000:3000 node:18 app
```

Common Flags

- `-d` : Detached mode (background)
- `-i` (`--interactive`): Keep STDIN open — send input to the container
- `-t` (`--tty`): Allocate a pseudo-terminal — gives you a proper shell prompt
- `-it` : Combined — use together for an interactive shell session
- `-p` : Port mapping `host:container`
- `-e KEY=value` : Single environment variable
- `--env-file .env` : Load all vars from file — preferred for secrets (add `.env` to `.dockerignore`)
- `--name` : Container name
- `-v myvolume:/data` : Named volume mount
- `-v $(pwd):/app` : Bind mount (host path → container)

docker → Managing Containers

```
# List running / all containers
docker ps
docker ps -a

# Stop / start / restart
docker stop <id>
docker start <id>
docker restart <id>

# Remove container / image
docker rm <id>
docker rmi <image_id>

# View logs (follow)
docker logs -f <id>
```

```
# Enter a running container
docker exec -it <id> bash

# Run a one-off command
docker exec <id> cat /etc/os-release

# Inspect full container metadata (JSON)
docker inspect <id>

# Copy files in/out of container
docker cp <id>:/app/log.txt ./log.txt
docker cp ./config.json <id>:/app/

# Clean up stopped containers + dangling images
docker system prune
```

docker → Images VS Containers

> 📖 *An image is an immutable blueprint. A container is a running (or stopped) instance of that image → with its own isolated writable layer on top.*

	Image	Container
What it is	Immutable blueprint / template	Running (or stopped) instance of an image
Created by	`docker build` or `docker pull`	`docker run`
Listed by	`docker images`	`docker ps -a`
Removed by	`docker rmi`	`docker rm`
Stored	Local image cache or registry	Writable layer on host filesystem

- Multiple containers can run from the same image
  - each gets its own isolated writable layer on top of the shared read-only image layers.

docker → Docker Commands Summary

Command	Description
`docker run`	Create and start container
`docker ps`	List running containers
`docker images`	List images
`docker pull`	Download image
`docker build`	Build image from Dockerfile
`docker push`	Upload image to registry
`docker exec -it`	Enter running container
`docker inspect`	Show container/image details
`docker logs -f`	Follow container logs
`docker stop` / `docker rm`	Stop / remove container
`docker system prune`	Clean up unused resources

Volumes

- Managed by Docker
  - best for production

```
# Create & use a named volume
docker volume create mydata
docker run -v mydata:/data img

# List / inspect / remove
docker volume ls
docker volume inspect mydata
docker volume rm mydata
```

- Data lives at `/var/lib/docker/volumes/` on host.
- **Survives** container deletion.

Bind Mounts

- Map a host directory
  - best for dev

```
# Mount current dir into container
docker run \
  -v $(pwd):/app \
  -w /app \
  python:3.11 python app.py
```

- Changes on host are **instantly visible** inside container.
- Ideal for live reload during development.

tmpfs Mounts

- In-memory only
  - best for secrets / temp files

```
docker run \
  --tmpfs /tmp:size=100m \
  --tmpfs /run \
  myimage
```

- Never written to disk.
- **Destroyed** when container stops.
- Fast for high-frequency temp I/O.

- Automatic DNS between containers on same network

Network Drivers

Driver	Use Case
bridge	Default — containers on same host communicate
host	Share host network stack (no isolation)
overlay	Multi-host (Swarm / K8s)
none	All networking disabled
macvlan	Container needs its own MAC/IP on LAN

Common Operations

```
# Create a custom bridge network
docker network create mynet

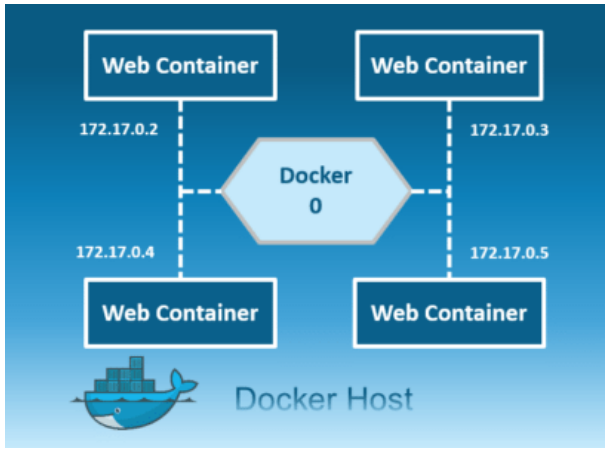
# Run containers on same network
# (reach each other by container name)
docker run --network mynet --name app myimage
docker run --network mynet --name db postgres

# List / inspect / remove networks
docker network ls
docker network inspect mynet
docker network rm mynet
```

- Port mapping: `-p 8080:80`
  - host port 8080 → container port 80

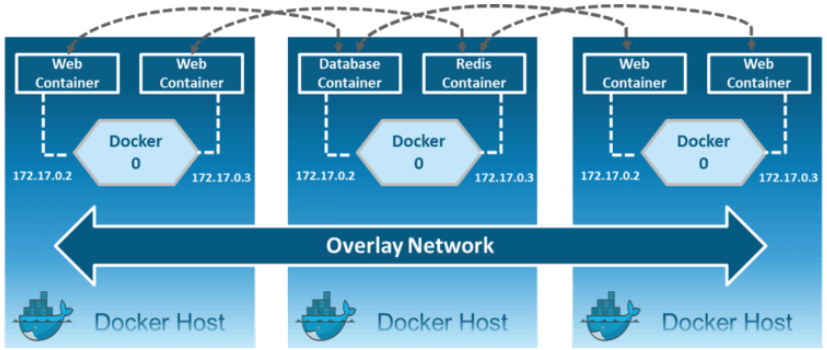
Bridge

- containers share Docker0 virtual switch



Overlay

- spans multiple Docker hosts (Swarm)



Public Registry → Docker Hub

- Default registry for all `docker pull / push`
- *Pull/push workflow covered in the Docker Hub slide*
- Free public repositories
- Official + community images
- URL: `docker.io/username/repo:tag`

Other Public Registries

- **GitHub Container Registry** (`ghcr.io`)
- **Google Artifact Registry** (`gcr.io`)
- **AWS ECR Public** (`public.ecr.aws`)

Private Registries

Registry	Provider
Harbor	Open source, self-hosted
AWS ECR	Amazon
Azure ACR	Microsoft
Google GAR	Google

Run your own:

```
docker run -d -p 5000:5000 registry:2

docker tag myapp localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
docker pull localhost:5000/myapp:1.0
```

Named vs Anonymous Volumes

```
# Named volume (reusable, inspectable)
docker run -v mydb:/var/lib/mysql mysql

# Anonymous volume (auto-named, harder to manage)
docker run -v /var/lib/mysql mysql
```

Share Data Between Containers

```
# Create shared volume
docker volume create shared

# Mount same volume in two containers
docker run -v shared:/data --name writer img
docker run -v shared:/data --name reader img
```

Backup & Restore Pattern

```
# Backup: tar volume contents via temp container
docker run --rm \
  -v mydata:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/mydata.tar.gz /data

# Restore
docker run --rm \
  -v mydata:/data \
  -v $(pwd):/backup \
  alpine tar xzf /backup/mydata.tar.gz -C /
```

Setup Steps

1. Install **Docker Desktop**
2. Install Dev Containers extension in VS Code
3. Create `.devcontainer/devcontainer.json` in your project
4. Press `Ctrl` + `Shift` + `P` → **"Dev Containers: Open Folder in Container"**
5. VS Code reconnects inside the container

```
{
  "name": "python-dev-env",
  "image": "python:3.11-alpine",
  "extensions": [
    "ms-python.python",
    "ms-python.vscode-pylance",
    "streetsidesoftware.code-spell-checker"
  ],
  "settings": {
    "python.defaultInterpreterPath": "/usr/local/bin/python"
  },
  "forwardPorts": [5000],
  "postCreateCommand": "pip install -r requirements.txt"
}
```

- Full IDE (extensions, terminal, debugger) runs inside the container
  - no local installs needed
- Teammates clone the repo and get an identical dev environment
- no "works on my machine" issues.

docker → Docker Compose

Define and run **multi-container** applications with a single file

```yaml
# version: field is optional in modern Compose V2
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db/mydb
    volumes:
      - ./app:/app           # bind mount for dev
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: pass

volumes:
  pgdata:    # named volume — persists across restarts
```

Essential Commands

```bash
# Start all services (detached)
docker-compose up -d

# Build and start
docker-compose up -d --build

# Stop and remove containers
docker-compose down

# Stop and remove + volumes
docker-compose down -v

# Follow logs for all services
docker-compose logs -f

# Follow logs for one service
docker-compose logs -f web

# Run one-off command in service
docker-compose exec web bash

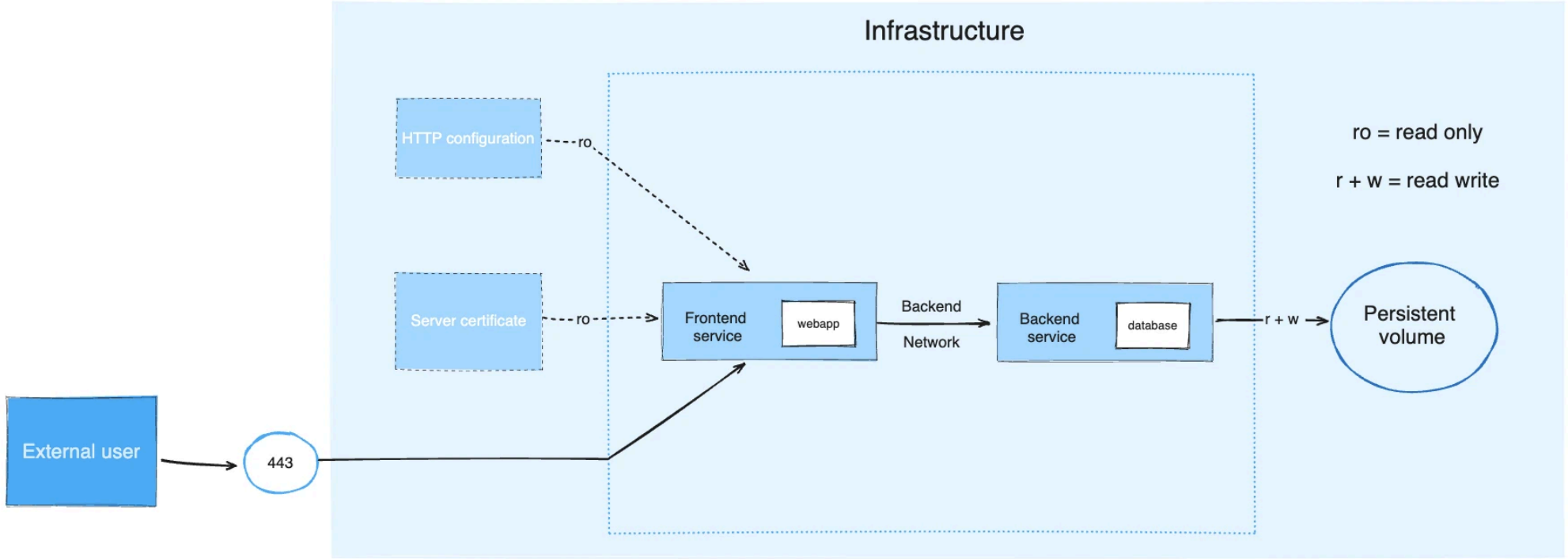
# Check status
docker-compose ps
```

Compose defines five core building blocks:

- **Services**
  - containers running the same image (scaled as replicas)
- **Networks**
  - IP routes between services; containers reach each other by service name
- **Volumes**
  - persistent filesystem mounts shared across services
- **Configs**
  - runtime configuration files injected into containers as mounts
- **Secrets**
  - sensitive data (certificates, passwords) injected securely as file mounts

- A **Project** groups all resources under one name (`docker compose -p myproject up`)
  - run the same `compose.yaml` twice on the same host with no conflicts.

> `compose.yaml` is the canonical filename (preferred over `docker-compose.yml`)



> *frontend + backend services on separate networks, with volumes, configs, and secrets injected at runtime.*

- Multi-container apps have a startup race condition
  - if `web` starts before `redis` is ready, it crashes.
  - Health checks + `depends_on` solve this.

```yaml
services:
  web:
    build: .
    depends_on:
      redis:
        condition: service_healthy   # wait until healthy

  redis:
    image: redis:alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5
```

`depends_on` conditions

- `service_started` — container created (default)
- `service_healthy` — healthcheck passing
- `service_completed_successfully` — exited with code 0 *(init containers, DB migrations)*

- `redis-cli ping` returns `PONG` when Redis is ready.
  - then does `web` start - guaranteed correct startup order

> 📖 *Watch mode eliminates the stop → rebuild → restart loop during development. Compose monitors file changes and reacts automatically.*

```yaml
services:
  web:
    build: .
    develop:
      watch:
        - action: sync+restart
          path: .
          target: /code
          ignore:
            - requirements.txt
        - action: rebuild
          path: requirements.txt
```

```
docker compose up --build --watch
```

Watch Actions

Action	When to use
`sync`	Copy files - no restart needed
`sync+restart`	Code change - fast, no rebuild
`rebuild`	Dependency change - full rebuild

Watch mode is a **development feature**

📖 *Native Docker clustering and orchestration tool*

- *groups multiple hosts into a single swarm, running containers as replicated services with built-in load balancing and failover.*

```
# Initialize swarm on manager node
docker swarm init

# Join worker node (use token from init output)
docker swarm join --token <token> <manager-ip>:2377

# Deploy a replicated service
docker service create \
  --name webserver \
  --replicas 3 \
  -p 80:80 \
  nginx:alpine

# List services / scale
docker service ls
docker service scale webserver=5
```

Swarm vs Kubernetes

	Swarm	Kubernetes
Complexity	Low	High
Features	Basic	Full
Setup	Minutes	Hours
Scale	Small–Medium	Any
Ecosystem	Docker native	Industry standard

Security

- Keep images minimal
  - smaller attack surface
- Run as **non-root** user (`USER appuser` in Dockerfile)
- Update base images regularly (patch CVEs)
- Use `.dockerignore` to exclude secrets & build artifacts:

```
.git
.env
*.log
node_modules
__pycache__
```

Efficiency

- Multi-stage builds
  - strip dev tools from final image
- Order Dockerfile instructions
  - stable layers first
  - changing last (maximizes cache hits)
- Combine `RUN` commands: `apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/*`
- Use `alpine` or `slim` base images
- Pin versions: `FROM python:3.11.9-slim` not `python:latest`

1 Hello World

```
docker run hello-world
```

2 Ubuntu Interactive

```
docker run -it ubuntu:latest bash # Start an interactive Ubuntu container
apt-get update && apt-get install -y vim # install Vim inside container
vim --version # check version
exit # Exit the container (container stops, data lost – ephemeral!)
```

3 Run Web Server

```
docker run -d -p 8080:80 nginx:alpine # Open http://localhost:8080 in browser
```

1 Create `Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY index.html .
EXPOSE 8000
CMD ["python", "-m", "http.server", "8000"]
```

2 Create `index.html` → `<h1>Hello from Docker!</h1>`

3 Build and run:

```
docker build -t my-python-app .
docker run -d -p 8000:8000 my-python-app
docker logs $(docker ps -q --filter name=my-python-app)
```

docker → Container Immutability

📖 *Images are immutable snapshots.*

- *Changing source files on the host does **not** affect a running container — it runs from the snapshot taken at build time.*
- *To update the app, rebuild the image and replace the container.*

Why host changes don't appear

```
docker build -t myapp .
docker run -d -p 3000:3000 --name myapp myapp

# Edit app.py on host → refresh browser
# → nothing changes in the container
# Container is isolated from host source files
```

Correct update workflow

```
# 1. Rebuild with changes
docker build -t myapp:v2 .

# 2. Replace the container
docker stop myapp && docker rm myapp
docker run -d -p 3000:3000 --name myapp myapp:v2
```

> Rollback = run the previous image tag. Every container is a known, reproducible artifact.

- `my-python-app` image must be built first.

```
docker volume create mydata
docker run -d --name myapp -v mydata:/app/data -p 8000:8000 my-python-app
docker exec myapp sh -c "echo hello > /app/data/test.txt"
docker stop myapp && docker rm myapp
```

- Re-run
  - data survives container deletion:

```
docker run -d --name myapp -v mydata:/app/data -p 8000:8000 my-python-app
docker exec myapp cat /app/data/test.txt
```

1 Create `docker-compose.yml`:

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
  redis:
    image: redis:alpine
```

2 Create `html/index.html` → `<h1>Hello from Compose!</h1>`

3 Start the app:

```
docker-compose up -d # Visit http://localhost:8080
docker-compose logs -f web
docker-compose down
```

Connect two containers over a custom network

- no `-p` port mapping needed between them.

```
# 1. Create a custom bridge network
docker network create mynet
# 2. Start a Redis container on that network
docker run -d --network mynet --name myredis redis:alpine
# 3. Start a Python container on the same network --> reaches Redis by container name "myredis"
docker run -it --network mynet python:3.11-slim bash
```

Inside the Python container

```
# Install redis client and connect
pip install redis -q
python -c "
import redis
r = redis.Redis(host='myredis', port=6379)
r.set('greeting', 'Hello from Docker!')
print(r.get('greeting')) "
```

Clean up

```
docker stop myredis && docker rm myredis
docker network rm mynet
```

docker → Common Issues & Solutions

Issue	Solution
Permission denied	Use `sudo` or add user to docker group
Port already in use	Find with `docker ps`, stop with `docker stop <id>`, or use different host port
Image not found	Pull first (`docker pull`)
Container exits immediately	Use `-it` or check CMD
`exec format error`	Image built for wrong CPU architecture → rebuild with `--platform linux/amd64`
Disk space full	`docker system prune`

References

- Docker Documentation
- Docker Hub
- Dockerfile Reference
- Docker Installation Guide
- Dev Containers (VS Code)
- Docker Compose Reference